

Day 40 — Line-by-Line Syntax Explanation

day40_weight_init_batchnorm.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 6, Day 5 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula (variance-preserving init scales, batch norm's forward/backward math), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the extended training work cycle, and the script's own real output graphs.

MATH box legend: $\times$ = matrix multiply / times W, b = weights/bias of a layer z^T = transpose of z
 $\text{mean}[...]/\text{sum}[...] = \text{averaged/summed over the batch}$ m = batch size n_{in} = fan-in (inputs to a layer)

Segment 0 — Imports and Global Setup (lines 31–37)

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

np.random.seed(42)
```

Identical setup pattern to Days 36-39: **np** for all array math, **matplotlib.use("Agg")** before importing **pyplot** so charts render straight to PNG files instead of a GUI window, **make_blobs** imported directly for this lesson's final training-comparison dataset, and **np.random.seed(42)** fixing every subsequent "random" NumPy call for reproducibility.

Segment 1 — Shared Helpers: tanh, softmax, one-hot, loss (lines 44–69)

MATH

$$d/dz[\tanh(z)] = 1 - \tanh(z)^2 = 1 - a^2$$

```
def tanh(z):
    return np.tanh(z)

def dtanh(a):
    # a = tanh(z), already computed -- avoids recomputing tanh
    return 1 - a ** 2
```

This lesson switches its hidden-layer activation from every prior day's **sigmoid** to **tanh** — chosen specifically because tanh's symmetric range (-1 to 1) makes the SATURATION behavior in Segment 3's signal-propagation demo visually and numerically clean. **dtanh** follows the same already-computed-output pattern as every prior day's derivative helper: it takes **a** (tanh's OUTPUT), not **z**, avoiding a redundant **np.tanh** call during backprop.

```
def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)

def one_hot(y, num_classes):
    m = y.shape[0]
    encoded = np.zeros((m, num_classes))
    encoded[np.arange(m), y] = 1.0
    return encoded

def cross_entropy_loss(probs, y_onehot, eps=1e-9):
    probs = np.clip(probs, eps, 1 - eps)
    return -np.mean(np.sum(y_onehot * np.log(probs), axis=1))
```

Character-for-character identical to Day 39's **softmax**, **one_hot**, and **cross_entropy_loss** — today's lesson is about what happens BEFORE and DURING training (initialization, normalization), not about changing the output layer or loss function at all, so these three functions are simply reused unmodified.

Segment 2 — init_scale: The Four Initialization Strategies (lines 72–83)

MATH

naive_small: scale = 0.01 naive_large: scale = 1.0

xavier: scale = $\sqrt{1 / n_{in}}$ he: scale = $\sqrt{2 / n_{in}}$

```
def init_scale(strategy, n_in):
    if strategy == "naive_small":
        return 0.01
    elif strategy == "naive_large":
        return 1.0
    elif strategy == "xavier":
        return np.sqrt(1.0 / n_in)
    elif strategy == "he":
        return np.sqrt(2.0 / n_in)
    raise ValueError(f"unknown strategy: {strategy}")
```

A single function centralizing all four strategies this lesson compares, returning one SCALE number that later multiplies a standard-normal random draw (**rng.randn(...)** * **scale**). **naive_small** and **naive_large** are FIXED numbers, independent of layer width — this is exactly what makes them naive. **xavier** and **he** both depend on **n_in** (the layer's fan-in), which is what lets them adapt correctly to any layer width. The trailing **raise ValueError** with no matching **elif** is a defensive catch-all for a typo'd strategy name.

Segment 3 — propagate_signal and demo_init_stats (lines 90–143)

MATH

$$z = a \times W + b, W = \text{randn}(\dots) \times \text{scale}$$

$$\text{Var}(z) \approx n_{\text{in}} \times \text{Var}(a) \times \text{scale}^2 \text{ (why the loop's std changes layer to layer)}$$

```
def propagate_signal(n_layers, n_units, strategy,
                    use_bn=False, n_samples=200, seed=0):
    rng = np.random.RandomState(seed)
    a = rng.randn(n_samples, n_units)
    stds = []
    for _ in range(n_layers):
        scale = init_scale(strategy, n_units)
        W = rng.randn(n_units, n_units) * scale
        b = np.zeros(n_units)
        z = a @ W + b
        if use_bn:
            mu = z.mean(axis=0, keepdims=True)
            var = z.var(axis=0, keepdims=True)
            z = (z - mu) / np.sqrt(var + 1e-5)
        a = tanh(z)
        stds.append(a.std())
    return stds
```

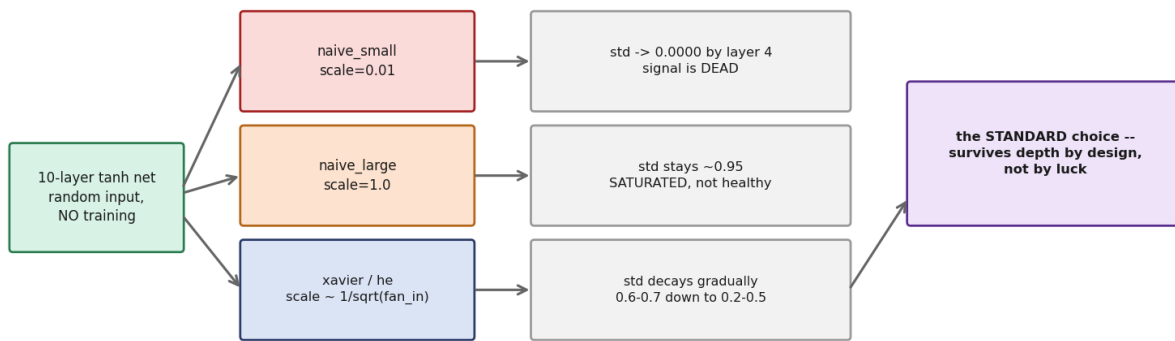
a = rng.randn(n_samples, n_units) starts with pure random noise as a stand-in input — no real dataset needed, since this function only measures what a given init SCALE does to signal shape, not what it learns. Each loop iteration builds a FRESH random weight matrix **W** (this is a diagnostic of the general strategy, not one fixed network), pushes the running activation **a** through it, optionally normalizes, applies **tanh**, and records only **a.std()** — a single number summarizing that layer's signal health. **use_bn** being **False** by default means this same function serves both Segment 3 (no batch norm) and Segment 4 (with batch norm) just by toggling one argument.

```
strategies = ["naive_small", "naive_large", "xavier", "he"]
results = {}
for strategy in strategies:
    stds = propagate_signal(n_layers=10, n_units=64, strategy=strategy)
    results[strategy] = stds
    rounded = [f"{s:.4f}" for s in stds]
    print(f"{strategy:14s} layer stds: {rounded}")
```

results is a plain dictionary mapping each strategy name to its list of 10 per-layer stds, built up one **propagate_signal** call per strategy so all four can be plotted together afterward. **f"{strategy:14s}"** left-pads the strategy name to 14 characters wide purely for aligned console output — a formatting detail, not a computation.

SYSTEM DIAGRAM

Four init strategies, one untrained 10-layer network (Segment 1)

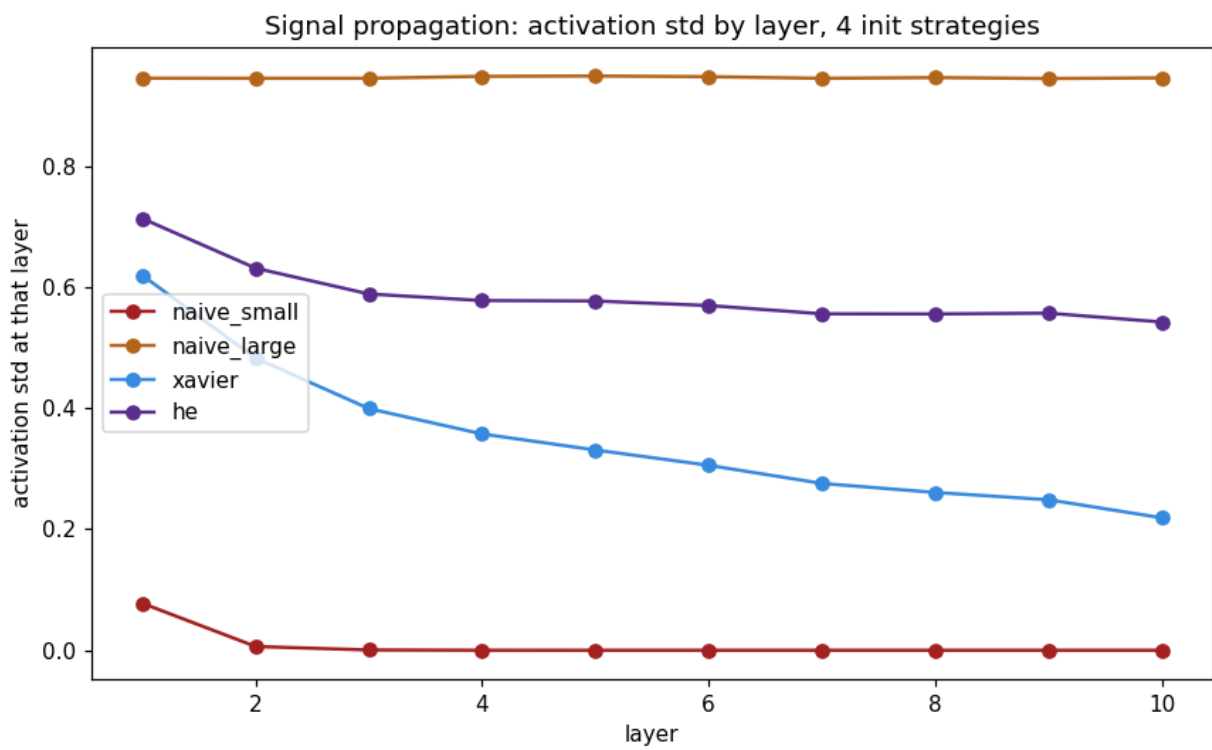


The four init strategies compared side by side, and what happens to a random signal after 10 layers of this exact loop, with no training.

```
fig, ax = plt.subplots(figsize=(8, 5))
colors = {"naive_small": "#a32020", "naive_large": "#b3651a",
          "xavier": "#378ADD", "he": "#5a2d8c"}
for strategy in strategies:
    ax.plot(range(1, 11), results[strategy], marker="o",
            label=strategy, color=colors[strategy], linewidth=1.6)
```

range(1, 11) labels the x-axis 1 through 10 (matching "layer 1" through "layer 10" rather than a 0-indexed 0-9, which would read confusingly on a chart meant for a human). Each of the four strategies gets its own fixed color from the **colors** dict, kept consistent with every other chart in this lesson that plots these same four strategies.

REAL GRAPH (script's own output)



Not a mockup — this is init_stats.png, this exact script rerun fresh: activation std by layer, all four strategies plotted together.

Segment 4 — demo_batchnorm_stats (lines 150–186)

```
all_stds = {}
for strategy in ["naive_small", "naive_large"]:
    stds_no_bn = propagate_signal(10, 64, strategy, use_bn=False)
    stds_bn = propagate_signal(10, 64, strategy, use_bn=True)
    all_stds[(strategy, False)] = stds_no_bn
    all_stds[(strategy, True)] = stds_bn
```

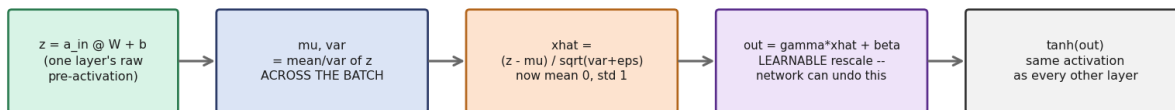
Only the two BAD strategies (naive_small, naive_large) are re-tested here — xavier and he don't need rescuing, so this demo focuses specifically on whether batch norm can fix the two broken cases from Segment 3. **all_stds** uses a TUPLE as a dictionary key (**(strategy, False)**) — a valid, hashable Python pattern for indexing by more than one property at once, avoiding four separate dictionaries.

```
fig, ax = plt.subplots(figsize=(8, 5))
styles = {
    ("naive_small", False): ("a32020", "--", "naive_small, no BN"),
    ("naive_small", True): ("a32020", "-", "naive_small, + BN"),
    ("naive_large", False): ("b3651a", "--", "naive_large, no BN"),
    ("naive_large", True): ("b3651a", "-", "naive_large, + BN"),
}
for key, stds in all_stds.items():
    color, linestyle, label = styles[key]
    ax.plot(range(1, 11), stds, marker="o", color=color,
            linestyle=linestyle, label=label, linewidth=1.6)
```

styles maps the same tuple keys used to build **all_stds** to a (color, linestyle, label) triple — dashed lines ("--") for the no-BN versions, solid lines ("-") for the with-BN versions, same color per strategy either way — so the chart visually groups by strategy while distinguishing before/after batch norm by line style alone.

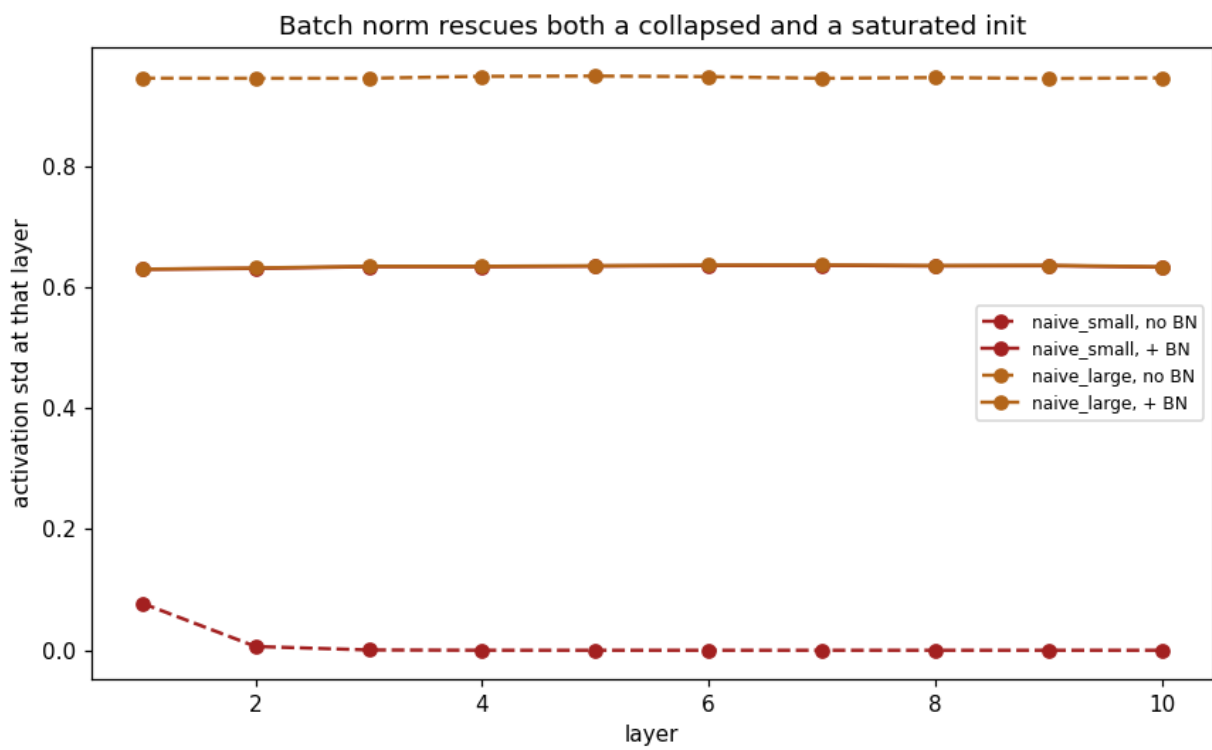
SYSTEM DIAGRAM

Batch normalization: one extra step inserted before every activation (Segment 2)



The five-step pipeline batch normalization inserts before every hidden layer's activation — exactly the branch toggled by `use_bn=True` inside `propagate_signal` above.

REAL GRAPH (script's own output)



Not a mockup — this is batchnorm_stats.png, this exact script rerun fresh: both a collapsed and a saturated init, rescued to nearly identical stable curves once batch norm is applied.

Segment 5 — DeepNet.__init__ (lines 198–212)

```
class DeepNet:
    def __init__(self, layer_sizes, init="xavier",
                  use_bn=False, lr=0.5, seed=42):
        rng = np.random.RandomState(seed)
        self.Ws, self.bs = [], []
        self.n_layers = len(layer_sizes) - 1
        for i in range(self.n_layers):
            n_in, n_out = layer_sizes[i], layer_sizes[i + 1]
            scale = init_scale(init, n_in)
            self.Ws.append(rng.randn(n_in, n_out) * scale)
            self.bs.append(np.zeros(n_out))
```

Unlike Day 39's **SoftmaxNet**, which stored exactly two weight matrices as two named attributes (**self.W1**, **self.W2**), **DeepNet** stores an arbitrary NUMBER of them in the LIST **self.Ws**, built by looping over **layer_sizes** — **len(layer_sizes) - 1** because a list of N layer widths defines exactly N-1 weight matrices between them. Each layer's own **n_in** is passed into **init_scale** individually, so a network with layers of different widths still gets correctly-scaled weights at every single layer.

MATH

$\gamma^{(l)} = 1, \beta^{(l)} = 0$ at init (identity transform, one pair per hidden layer)

```
self.n_hidden = self.n_layers - 1
self.use_bn = use_bn
if use_bn:
    self.gammas = [np.ones(layer_sizes[i + 1])
                    for i in range(self.n_hidden)]
    self.betas = [np.zeros(layer_sizes[i + 1])
                   for i in range(self.n_hidden)]
self.lr = lr
```

self.n_hidden subtracts one more from **n_layers** because the LAST weight matrix maps the final hidden layer to the OUTPUT (softmax) layer, which never gets batch-normalized in this lesson — only the **n_hidden** HIDDEN layers do. **self.gammas** and **self.betas** are built with LIST COMPREHENSIONS, one gamma/beta pair per hidden layer, initialized to 1/0 (the identity transform) — these lists only exist at all when **use_bn=True**, saved as regular Python attributes just like **self.Ws**.

Segment 6 — DeepNet.forward (lines 214–233)

MATH

$$z = a_{in} \times W + b$$

if use_bn: $xhat = (z - \mu) / \sqrt{var + \epsilon}$, $z = \gamma \times xhat + \beta$

$$a_{out} = \tanh(z)$$

```
def forward(self, X):
    self.cache = []
    a = X
    for i in range(self.n_hidden):
        z = a @ self.Ws[i] + self.bs[i]
        if self.use_bn:
            mu = z.mean(axis=0, keepdims=True)
            var = z.var(axis=0, keepdims=True)
            xhat = (z - mu) / np.sqrt(var + 1e-5)
            z = self.gammas[i] * xhat + self.betas[i]
            a_out = tanh(z)
            self.cache.append(dict(a_in=a, z=z, mu=mu,
                                   var=var, xhat=xhat,
                                   a_out=a_out))
        else:
            a_out = tanh(z)
            self.cache.append(dict(a_in=a, a_out=a_out))
        a = a_out
```

The core generalization from Day 39's fixed 2-line forward pass to an arbitrary-depth LOOP over **self.n_hidden** layers. **self.cache** is a LIST of dictionaries, one per layer, saving exactly what that layer will need again during **backward()** — the with-batch-norm branch saves five values (**a_in**, **z**, **mu**, **var**, **xhat**, **a_out**) versus the without-batch-norm branch's two (**a_in**, **a_out**), directly reflecting how much more backward-pass bookkeeping batch norm requires. **a = a_out** at the end of every loop iteration feeds this layer's output forward as the NEXT layer's input, exactly like a chain.

```
z_out = a @ self.Ws[-1] + self.bs[-1]
probs = softmax(z_out)
self.cache.append(dict(a_in=a, probs=probs))
return probs
```

self.Ws[-1] and **self.bs[-1]** use NEGATIVE INDEXING to grab the LAST weight matrix in the list — the output layer's weights — regardless of how many hidden layers came before it. This output layer never gets batch-normalized (mirroring Day 39's SoftmaxNet, where the output layer is always plain **softmax(z_out)**) — the final cache entry stores just enough (**a_in**, **probs**) for the output layer's own backward step.

Segment 7 — DeepNet.backward (lines 235–272)

MATH

$dz_{out} = probs - y_{onehot}$ (Day 39's result, unchanged by depth)

```
def backward(self, y_onehot):
    m = y_onehot.shape[0]
    out = self.cache[-1]
    dz = out["probs"] - y_onehot
    dws, dbs = [None] * self.n_layers, [None] * self.n_layers
    dgammas = [None] * self.n_hidden if self.use_bn else None
    dbetas = [None] * self.n_hidden if self.use_bn else None

    dws[-1] = out["a_in"].T @ dz / m
    dbs[-1] = dz.mean(axis=0)
    da = dz @ self.Ws[-1].T
```

dz = out["probs"] - y_onehot is Day 39's exact softmax+cross-entropy simplification, unchanged by depth — read straight back out of the cache dictionary saved during **forward()**. **dWs** and **dbs** are pre-allocated as **[None] * self.n_layers** lists, filled in REVERSE order as the loop below walks backward through the network — the output layer's gradients (**dWs[-1]**, **dbs[-1]**) are computed first, before the loop even starts, exactly mirroring the order computation happens during real backpropagation.

```
for i in reversed(range(self.n_hidden)):
    c = self.cache[i]
    dz_act = da * dtanh(c["a_out"])
    if self.use_bn:
        xhat = c["xhat"]; mu = c["mu"]
        var = c["var"]; z = c["z"]
        gamma = self.gammas[i]
        dgammas[i] = np.sum(dz_act * xhat, axis=0)
        dbetas[i] = np.sum(dz_act, axis=0)
        dxhat = dz_act * gamma
        std_inv = 1.0 / np.sqrt(var + 1e-5)
```

reversed(range(self.n_hidden)) walks the hidden-layer indices backward (e.g. 5, 4, 3, 2, 1, 0 for 6 hidden layers) — the same direction real backpropagation always flows in. **dz_act = da * dtanh(c["a_out"])** is the gradient arriving at this layer's OUTPUT (after tanh), computed using this layer's own cached **a_out** — the running **da** variable is what carries the gradient from the layer AFTER this one, updated at the end of every loop iteration below.

MATH

$$dvar = \sum(dxhat \times (z - \mu) \times -0.5 \times (var + \epsilon)^{-3/2})$$
$$dmu = \sum(dxhat \times -std_inv) + dvar \times \text{mean}(-2(z - \mu))$$
$$dz_lin = dxhat \times std_inv + dvar \times 2(z - \mu) / m + dmu / m$$

```

dvar = np.sum(dxhat * (z - mu) * -0.5
              * std_inv ** 3, axis=0)
dmu = (np.sum(dxhat * -std_inv, axis=0)
      + dvar * np.mean(-2 * (z - mu), axis=0))
dz_lin = (dxhat * std_inv
          + dvar * 2 * (z - mu) / m
          + dmu / m)
else:
    dz_lin = dz_act

```

This is Segment 4's batch-norm mechanism, differentiated by hand: **dvar** and **dmu** exist specifically because **mu** and **var** are FUNCTIONS of the whole batch, not independent constants — each requires summing over every point's contribution (**np.sum(..., axis=0)**). The final **dz_lin** combines all three gradient paths (direct through xhat, indirect through var, indirect through mu) into the one gradient value that continues backward into the linear layer — parentheses wrap each multi-line expression here purely for readability, with no change in meaning.

MATH

$$dW^{(i)} = (1/m) \times a_{in}^T \times dz_{lin}$$

$$da^{(i-1)} = dz_{lin} \times W^{(i)T}$$

```

dws[i] = c["a_in"].T @ dz_lin / m
dbs[i] = dz_lin.mean(axis=0)
da = dz_lin @ self.ws[i].T

```

The same three-line pattern as every prior day's weight-gradient computation — **dws[i]**, **dbs[i]** — followed by computing **da**, the gradient to hand off to the NEXT (earlier) iteration of this loop as ITS incoming **da**, exactly how a chain passes a value from one link to the next.

```

for i in range(self.n_layers):
    self.ws[i] -= self.lr * dws[i]
    self.bs[i] -= self.lr * dbs[i]
if self.use_bn:
    for i in range(self.n_hidden):
        self.gammas[i] -= self.lr * dgammas[i]
        self.betas[i] -= self.lr * dbetas[i]

```

Two SEPARATE update loops: one over all **n_layers** weight matrices (always runs), one over just the **n_hidden** gamma/beta pairs (only runs when batch norm is enabled) — the same plain gradient-descent update rule (**param -= lr * gradient**) applied uniformly to every learnable parameter in the network, regardless of which layer or which kind of parameter it is.

Segment 8 — DeepNet.train and DeepNet.accuracy (lines 274–285)

```
def train(self, X, y, epochs, num_classes):
    y_onehot = one_hot(y, num_classes)
    losses = []
    for _ in range(epochs):
        probs = self.forward(X)
        losses.append(cross_entropy_loss(probs, y_onehot))
        self.backward(y_onehot)
    return losses
```

Deliberately SIMPLER than Day 39's **train()** — no mini-batching option here at all, every epoch is one full-batch forward+backward call. This is intentional: today's lesson isolates initialization and batch norm as the variables under test, so batching strategy (Day 39's topic) is deliberately held fixed and out of the comparison.

```
def accuracy(self, X, y):
    probs = self.forward(X)
    return (probs.argmax(axis=1) == y).mean()
```

Identical to Day 39's **accuracy** method — **argmax(axis=1)** picks the most-confident class per row, compared directly against the integer labels **y** and averaged into a single accuracy number.

Segment 9 — make_dataset and demo_training_comparison (lines 292–359)

```
def make_dataset():
    return make_blobs(n_samples=300, centers=3, n_features=2,
                      cluster_std=1.8, random_state=42)
```

The exact same dataset generation call as Day 39's **make_multiclass_dataset** — 300 points, 3 classes, **random_state=42** — reused deliberately so this lesson's training results are directly comparable to Day 39's, with only the network architecture and initialization changing.

```
layer_sizes = [2] + [32] * 6 + [3]

results = {}
for init in ["naive_small", "naive_large", "xavier", "he"]:
    net = DeepNet(layer_sizes, init=init, use_bn=False,
                  lr=0.5, seed=42)
    losses = net.train(X, y, epochs=300, num_classes=3)
    acc = net.accuracy(X, y)
    results[init] = losses
```

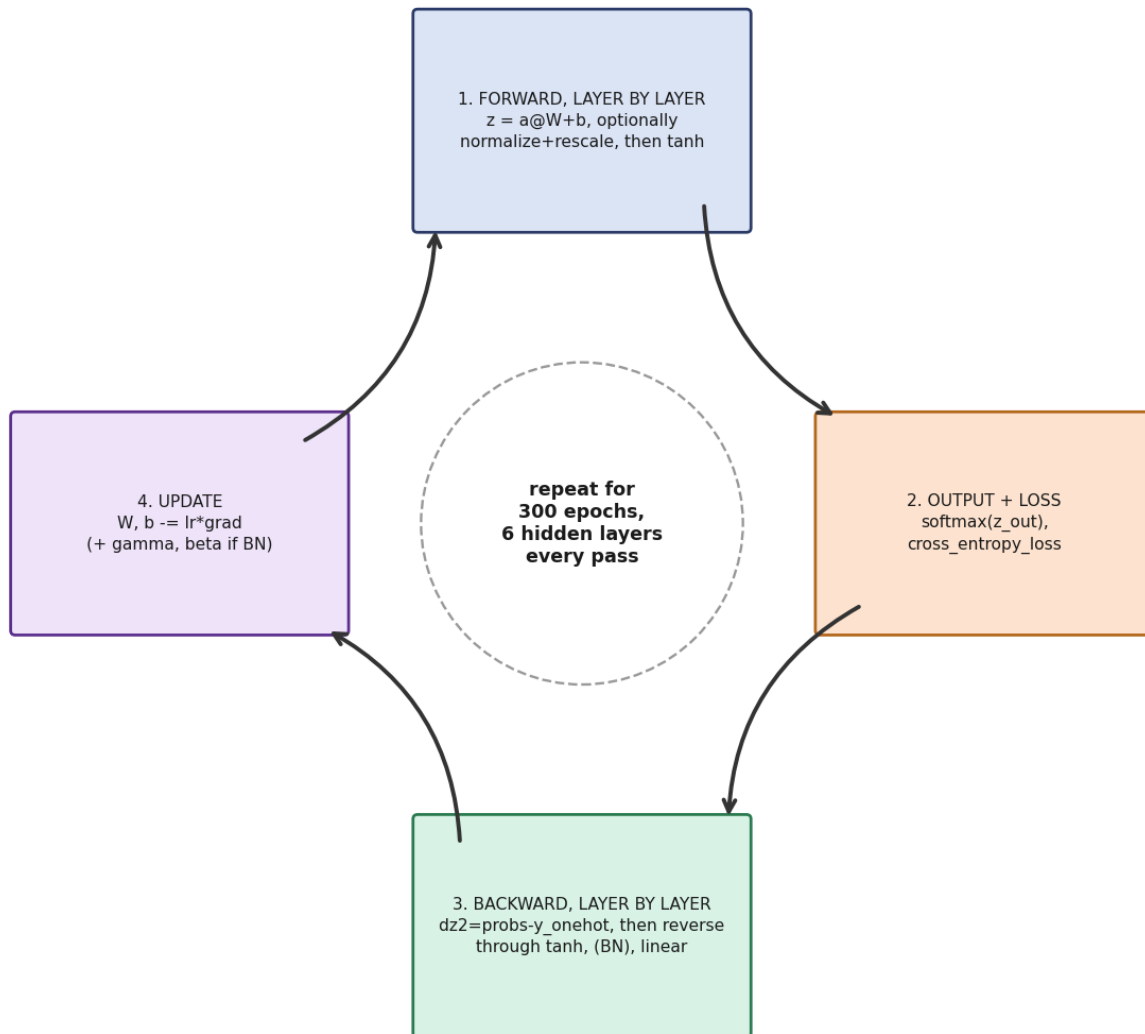
[2] + [32] * 6 + [3] builds the `layer_sizes` list via PLAIN LIST ARITHMETIC: **[32] * 6** repeats the single-element list **[32]** six times, then **+** concatenates lists (not numeric addition) to produce **[2, 32, 32, 32, 32, 32, 32, 3]** — 6 hidden layers of 32 units, bracketed by the 2D input and 3-class output. Every one of the four initialization strategies trains its OWN fresh **DeepNet** with identical **seed=42** starting weights (per strategy) and an identical 300-epoch budget, making the comparison fair.

```
bn_results = {}
for init in ["naive_small", "naive_large"]:
    net = DeepNet(layer_sizes, init=init, use_bn=True,
                  lr=0.5, seed=42)
    losses = net.train(X, y, epochs=300, num_classes=3)
    acc = net.accuracy(X, y)
    bn_results[init] = losses
```

The same two loop bodies as above, with only **use_bn=True** changed, run again for just the two initializations worth rescuing — producing a SEPARATE dictionary (**bn_results**) so the with/without comparison chart below can plot both versions of each strategy.

SYSTEM DIAGRAM

The training work cycle, generalized to N layers (+ optional BN)



This segment's `train()` loop as a work cycle, generalized to N layers with an optional batch-norm step in both the forward and backward pass.

```
ns_final = bn_results["naive_small"][-1]
nl_final = bn_results["naive_large"][-1]
print(f"...loss drops to {ns_final:.4f}...")
```

Notice this print statement reads its number DIRECTLY from the just-computed **bn_results** list (**[-1]** for the final epoch's loss) rather than a hardcoded value typed in by hand — this guarantees the narrative text always matches whatever this exact run actually produced, even if a different random seed or architecture change shifted the number later.


```

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
colors = {"naive_small": "#a32020", "naive_large": "#b3651a",
          "xavier": "#378ADD", "he": "#5a2d8c"}
for init, losses in results.items():
    axes[0].plot(losses, label=init, color=colors[init],
                  linewidth=1.4)
axes[0].set_ylim(0, 2)

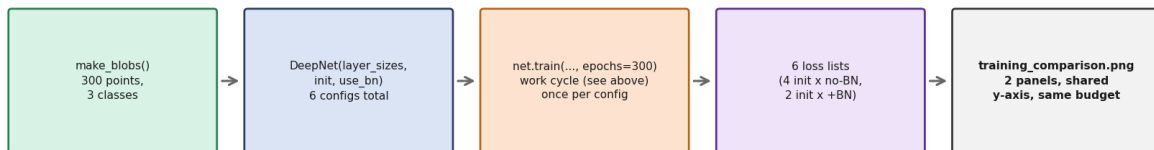
for init, losses in bn_results.items():
    axes[1].plot(losses, label=f"{init} + BN",
                  color=colors[init], linewidth=1.4)
axes[1].set_ylim(0, 2)

```

plt.subplots(1, 2, ...) creates a 1-ROW, 2-COLUMN grid of axes — **axes[0]** (left, no batch norm) and **axes[1]** (right, with batch norm) — sharing the SAME **set_ylim(0, 2)** range on both, which is what makes it visually honest to compare naive_small's flatlined loss on the left against its rescued curve on the right at a glance.

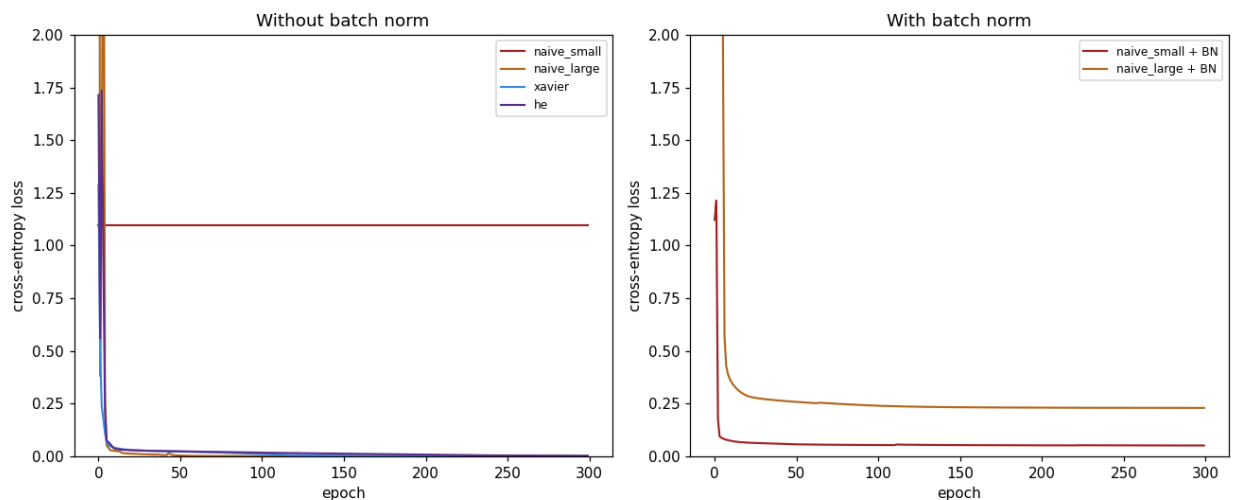
SYSTEM DIAGRAM

Segment 3 pipeline: how code + math becomes the final comparison chart



Zooming out: the full pipeline this segment runs end to end, from the shared dataset through six independent training runs to the final comparison chart.

REAL GRAPH (script's own output)



Not a mockup — this is training_comparison.png, this exact script rerun fresh. Left: all four inits without batch norm (naive_small flatlines at the top). Right: naive_small and naive_large with batch norm added.

Segment 10 — `note_on_pytorch`, `main`, and the Entry-Point Guard (lines 362–391)

```
nn.init.xavier_uniform_(layer.weight)
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')
nn.BatchNorm1d(num_features)
```

`note_on_pytorch()` is just `print()` calls containing plain text — not executable code, and PyTorch is never imported anywhere in this file, exactly like Day 39's equivalent function. These are reference lines mapping each hand-written mechanism (Segments 2, 5-7) to its PyTorch equivalent.

```
def main():
    demo_init_stats()
    demo_batchnorm_stats()
    demo_training_comparison()
    note_on_pytorch()

if __name__ == "__main__":
    main()
```

The same overall structure as every prior day: one `main()` calling every demo in the exact order they should run, guarded by the standard `if __name__ == "__main__":`: check so nothing auto-runs if **DeepNet** or **init_scale** is ever imported from this file elsewhere.

Quick Syntax Glossary — New or Reinforced Constructs

`dict with a tuple key` — `all_stds[(strategy, use_bn)] = stds` uses a (str, bool) tuple as a single dictionary key, indexing by two properties at once without needing four separate dictionaries.

`list of dicts as a per-layer cache` — `self.cache.append(dict(...))` builds one dictionary per layer inside a list -- each layer's forward-pass values are looked back up by index (`self.cache[i]`) during `backward()`.

`[value] * n list repetition` — `[32] * 6` repeats a single-element list six times; concatenating with `+` (not numeric addition) builds `layer_sizes = [2, 32, 32, 32, 32, 32, 32, 3]` without writing six literal 32s.

`reversed(range(n))` — Walks a range backward (n-1, n-2, ..., 0) -- used in `backward()` so the hidden-layer loop processes layers in the same reverse order real backpropagation flows in.

`self.ws[-1] / self.bs[-1]` — Negative indexing grabs the LAST element of a list -- here, the output layer's weights, regardless of how many hidden layers came before it in a variable-depth network.

`[None] * n pre-allocation` — `dWs = [None] * self.n_layers` creates a fixed-length list of placeholders, filled in by index as the backward loop computes each layer's gradient, rather than using `.append()`.

`conditional list/expr = ... if ... else None` — `dgammas = [None] * n if self.use_bn else None` -- a one-line conditional that creates a list only when batch norm is active, avoiding an if/else block.

`multi-line expression in parentheses` — Wrapping `(a + b + c)` across several lines inside parentheses is valid Python with no line-continuation backslash needed -- used to keep long gradient formulas readable.

`f-string reading a just-computed variable` — `print(f"...{ns_final:.4f}...")` reads its number directly from a variable computed moments earlier in the same run, rather than a hardcoded value -- guaranteeing the printed narrative always matches the real output.

`plt.subplots(1, 2, ...)` — Creates a 1-row, 2-column grid of subplots at once, returned as an indexable array (`axes[0]`, `axes[1]`) -- used here to place the without-BN and with-BN charts side by side.

End of Day 40 syntax reference. For the initialization and batch normalization concepts these lines implement, see the main Day 40 study guide PDF.